

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	Date:	Enrolment No:92301733043

Aim: To study different word embedding for representation of textual data in vectorized form

IDE: Google Colab

Theory:

Word Embedding is an approach for representing words and documents. Word Embedding or Word Vector is a numeric vector input that represents a word in a lower-dimensional space. It allows words with similar meanings to have a similar representation.

Word Embeddings are a method of extracting features out of text so that we can input those features into a machine learning model to work with text data. They try to preserve syntactical and semantic information. The methods such as Bag of Words (BOW), CountVectorizer and TFIDF rely on the word count in a sentence but do not save any syntactical or semantic information. In these algorithms, the size of the vector is the number of elements in the vocabulary. We can get a sparse matrix if most of the elements are zero. Large input vectors will mean a huge number of weights which will result in high computation required for training. Word Embeddings give a solution to these problems.

Need for Word Embedding?

- To reduce dimensionality
- To use a word to predict the words around it.
- Inter-word semantics must be captured.

How are Word Embeddings used?

- They are used as input to machine learning models.
Take the words → Give their numeric representation → Use in training or inference.
- To represent or visualize any underlying patterns of usage in the corpus that was used to train them.

1. Bag of Words

Bag of Words (BOW) is an algorithm that counts how many times a word appears in a document. Those word counts allow us to compare documents and gauge their similarities for applications like search, document classification, and topic modeling.

2. Tf-Idf

Tf-Idf is shorthand for term frequency-inverse document frequency. So, two things: term frequency and inverse document frequency. Term frequency (TF) is basically the output of the BoW model. For a specific document, it determines how important a word is by looking at how frequently it appears in the document. Term frequency measures the importance of the word. If a word appears a lot of times, then the word must be important. For example, if our document is “I am a cat lover. I have a cat named Steve. I feed a cat outside my room regularly,” we see that the words with the highest frequency are I, a, and cat. This agrees with our intuition that high term frequency = higher importance.

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	Date:	Enrolment No:92301733043

$TF(t) = (\text{Number of times term } t \text{ appears in a document}) / (\text{Total number of terms in the document})$

IDF used to calculate the weight of rare words across all documents. The words that occur rarely in the corpus have a high IDF score. However, it is known that certain terms, such as “I”, “a” may appear a lot of times but have little importance. Thus we need to weigh down the frequent terms while scaling up the rare ones

$IDF(t) = \log_e(\text{Total number of documents} / \text{Number of documents with term } t \text{ in it})$

Pre Lab Exercise:

1. Perform BoW vectorization of the corpus

CODE:

```
from sklearn.feature_extraction.text import CountVectorizer
# Sample corpus
corpus = ["I love machine learning", "Machine learning is fun", "I love coding"]
# BoW
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(corpus)
print("Vocabulary:", vectorizer.get_feature_names_out())
print("BoW Matrix:\n", X.toarray())
```

OUTPUT:

```
Vocabulary: ['coding' 'fun' 'is' 'learning' 'love' 'machine']
BoW Matrix:
[[0 0 0 1 1 1]
 [0 1 1 1 0 1]
 [1 0 0 0 1 0]]
```

2. Perform TF vectorization of the corpus

CODE:

```
from sklearn.feature_extraction.text import TfidfVectorizer
# TF only (disable IDF)
tf_vectorizer = TfidfVectorizer(use_idf=False)
X_tf = tf_vectorizer.fit_transform(corpus)
print("Vocabulary:", tf_vectorizer.get_feature_names_out())
print("TF Matrix:\n", X_tf.toarray())
```

OUTPUT:

```
Vocabulary: ['coding' 'fun' 'is' 'learning' 'love' 'machine']
TF Matrix:
[[0.          0.          0.          0.57735027 0.57735027 0.57735027]
 [0.          0.5         0.5         0.5         0.          0.5         ]
 [0.70710678 0.          0.          0.          0.70710678 0.          ]]
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	Date:	Enrolment No:92301733043

Program (Code):

To be attached with

1. Perform the TF-IDF vectorization of the corpus

CODE:

```
from sklearn.feature_extraction.text import TfidfVectorizer
import pandas as pd
corpus = [
    "I love machine learning",
    "Machine learning is amazing",
    "I love coding"
]
# Initialize vectorizer
vectorizer = TfidfVectorizer()
# Fit and transform corpus
X = vectorizer.fit_transform(corpus)
# Convert to array
tfidf_matrix = X.toarray()
# Feature names (words)
features = vectorizer.get_feature_names_out()
print("Features:", features)
print("TF-IDF Matrix:\n", tfidf_matrix)
df_tfidf = pd.DataFrame(tfidf_matrix, columns=features)
print(df_tfidf)
vectorizer = TfidfVectorizer(stop_words='english')
X = vectorizer.fit_transform(corpus)
```

Results:

```
Features: ['amazing' 'coding' 'is' 'learning' 'love' 'machine']
TF-IDF Matrix:
[[0.          0.          0.          0.57735027 0.57735027 0.57735027]
 [0.5628291   0.          0.5628291   0.42804604 0.          0.42804604]
 [0.          0.79596054 0.          0.          0.60534851 0.          ]]
   amazing  coding    is  learning    love  machine
0  0.000000  0.000000  0.000000  0.577350  0.577350  0.577350
1  0.562829  0.000000  0.562829  0.428046  0.000000  0.428046
2  0.000000  0.795961  0.000000  0.000000  0.605349  0.000000
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	Date:	Enrolment No:92301733043

Observation:

Word embedding is a technique used to convert textual data into numerical form so that it can be processed by machine learning models. Traditional methods like Bag of Words (BoW) and TF-IDF represent text using word frequency but fail to capture semantic and contextual meaning. These methods often produce high-dimensional sparse vectors.

Word embeddings help reduce dimensionality and capture relationships between words by placing similar words closer in vector space. While BoW and TF-IDF are simple and useful for basic tasks, advanced embedding methods provide better semantic understanding.

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	Date:	Enrolment No:92301733043

Post Lab Exercise:

Take three documents and find similarity using

documents = [

"I love machine learning",

"Machine learning is amazing",

"I enjoy coding"

]

- a. BoW vectorization

CODE:

```
from sklearn.metrics.pairwise import cosine_similarity
bow = CountVectorizer()
X_bow = bow.fit_transform(documents)
similarity_bow = cosine_similarity(X_bow)
print("BoW Similarity:\n", similarity_bow)
```

OUTPUT:

```
BoW Similarity:
[[1.         0.57735027 0.         ]
 [0.57735027 1.         0.         ]
 [0.         0.         1.         ]]
```

- b. TF Vectorization

CODE:

```
tf = TfidfVectorizer(use_idf=False)
X_tf = tf.fit_transform(documents)
similarity_tf = cosine_similarity(X_tf)
print("TF Similarity:\n", similarity_tf)
```

OUTPUT:

```
TF Similarity:
[[1.         0.57735027 0.         ]
 [0.57735027 1.         0.         ]
 [0.         0.         1.         ]]
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	Date:	Enrolment No:92301733043

c. TF-IDF Vectorization

CODE:

```
tfidf = TfidfVectorizer()
X_tfidf = tfidf.fit_transform(documents)
similarity_tfidf = cosine_similarity(X_tfidf)
print("TF-IDF Similarity:\n", similarity_tfidf)
```

OUTPUT:

```
TF-IDF Similarity:
[[1.          0.44333251  0.          ]
 [0.44333251  1.          0.          ]
 [0.          0.          1.          ]]
```

Comment over the answer obtained in each of the case.

BoW Similarity Analysis :

Doc 0 & Doc 1: Similarity of ~0.577 due to shared words ("machine", "learning").

Doc 0 & Doc 2; Doc 1 & Doc 2: Similarity of 0.0, indicating no shared words.

BoW focuses on word presence/frequency, treating documents as word collections.

TF Similarity Analysis :

Similarity scores are identical to BoW (~0.577 for Doc 0 & 1, 0.0 otherwise).

For this small corpus, TF behaves similarly to BoW after normalization.

TF-IDF Similarity Analysis :

Doc 0 & Doc 1: Similarity of ~0.443, lower than BoW/TF.

Reason: TF-IDF reduces importance of common words ("I", "machine", "learning") and highlights unique terms ("love", "amazing"), leading to a lower similarity score for shared but less distinctive terms.

Doc 0 & Doc 2; Doc 1 & Doc 2: Still 0.0 similarity.

TF-IDF is preferred for information retrieval as it weighs terms by their discriminative power.